

DATABASE DESIGN DOCUMENT (DDD)

Table of Contents

Chapter No.	Title
1	Introduction
2	Database Architecture
3	Database Design Principles
4	Database Schema Overview
5	PSP Ledger Database Design
6	NPCI Ledger Database Design
7	Bank Ledger Database Design
8	Audit Database Design
9	Data Flow
10	Constraints and Keys
11	Backup and Recovery
12	Future Enhancements
13	Conclusion

Chapter 1 – Introduction

1.1 Purpose

The Database Design Document (DDD) describes the database architecture of the APEX UPI system. It explains the database schema, table structures, primary keys, data ownership, and how different microservices store transaction information in PostgreSQL.

The document serves as a reference for developers, testers, and database administrators involved in implementing and maintaining the system.

1.2 Scope

This document covers the database implementation for:

- PSP Ledger Service
- NPCI Ledger Service
- Bank Ledger Service
- Audit Service

Each service stores its own data in PostgreSQL following the database-per-service principle.

1.3 Database Technology

Item	Value
Database	PostgreSQL
Version	PostgreSQL 16 (or installed version)
ORM	Spring Data JPA
Language	SQL
Connectivity	JDBC

Chapter 2 – Database Architecture

The system follows a decentralized database architecture where each microservice owns its corresponding table. This prevents tight coupling between services and improves scalability.

2.1 Overview

The HPE APEX UPI Payment Switch follows a **microservice-oriented database architecture** designed to ensure scalability, fault isolation, high availability, and maintainability. Instead of using a single shared database, each microservice owns and manages its own database table. This approach aligns with modern cloud-native application design principles, where services remain independent and communicate only through asynchronous Kafka events.

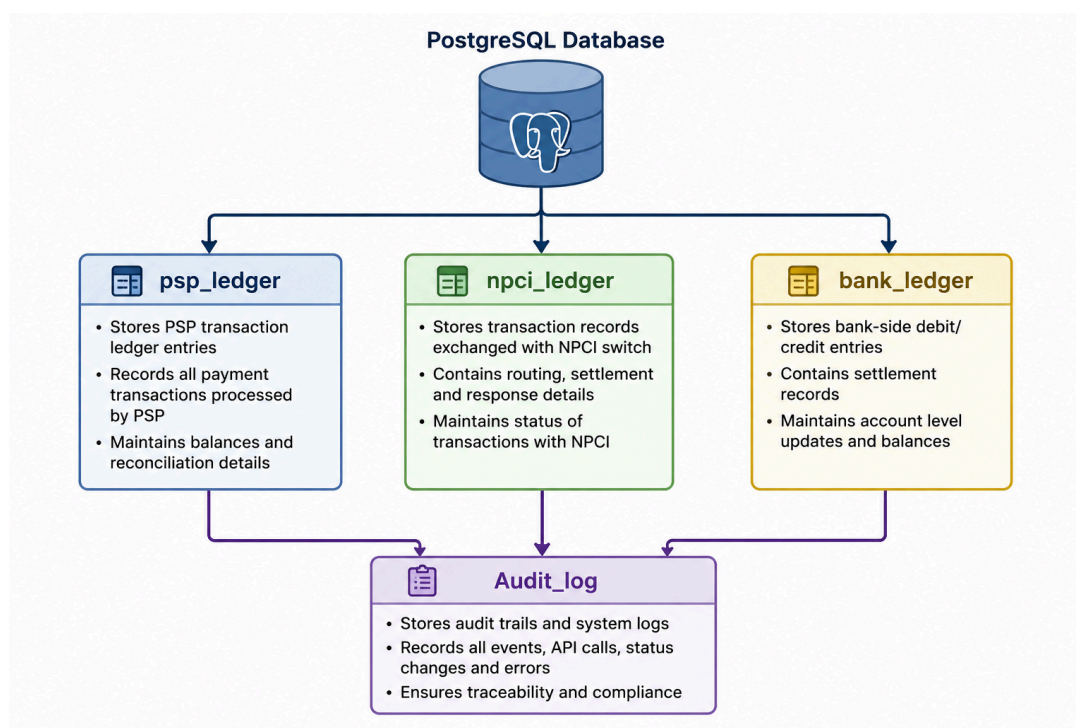
The database layer is implemented using **PostgreSQL**, an enterprise-grade relational database management system that provides ACID-compliant transactions, strong consistency, indexing support, and high-performance query execution. PostgreSQL serves as the persistent storage layer for all transaction, ledger, and audit information generated during UPI payment processing.

2.2 Database Architecture

The PostgreSQL database contains four logical tables, each managed by a dedicated microservice.

- **PSP Ledger Service** manages the **psp_ledger** table.
- **NPCI Ledger Service** manages the **npci_ledger** table.
- **Bank Ledger Service** manages the **bank_ledger** table.
- **Audit Service** manages the **audit_log** table.

All transaction updates are exchanged through Kafka topics rather than direct database communication. This event-driven architecture ensures loose coupling between services while maintaining data consistency across the system.



2.3 Database Components

PSP Ledger Database

The PSP Ledger database stores the initial transaction information received from the PSP Switch. Whenever a customer initiates a UPI payment, the PSP Switch validates the request and publishes a transaction event. The PSP Ledger Service consumes this event and stores the transaction details, including payer VPA, payee VPA, transaction amount, and transaction status.

NPCI Ledger Database

The NPCI Ledger database stores routing and verification information exchanged with the NPCI network. After transaction validation, the NPCI Router forwards the request for verification. The corresponding ledger service records the source bank, destination bank, verification status, and routing details for every transaction.

Bank Ledger Database

The Bank Ledger database maintains all financial accounting entries generated after successful debit and credit operations. For every successful UPI transaction, separate debit and credit records are created to maintain accurate accounting and simplify reconciliation. This ledger acts as the financial record of all completed banking operations.

Audit Database

The Audit database captures the complete lifecycle of every transaction processed by the system. It stores transaction events generated by all services, including payment initiation, verification, debit confirmation, credit confirmation, reversals, and failures. These records are essential for monitoring, troubleshooting, compliance, and transaction traceability.

2.4 Event-Driven Database Communication

The database architecture follows an **event-driven communication model** using Apache Kafka. Services never access another service's database table directly. Instead, each service publishes events after completing its operation, and the corresponding consumer updates its own database table.

For example:

1. The PSP Switch publishes a payment initiation event.
2. The PSP Ledger Service stores the transaction in **psp_ledger**.
3. The NPCI Router verifies the transaction and publishes a verification event.
4. The NPCI Ledger Service stores routing information in **npci_ledger**.
5. The CBS Service processes debit and credit operations and publishes confirmation events.
6. The Bank Ledger Service stores accounting entries in **bank_ledger**.
7. The Audit Service continuously listens to all Kafka topics and records every event in the **audit_log** table.

This asynchronous communication mechanism improves system throughput, enables independent service deployment, and minimizes dependencies between microservices.

2.5 Advantages of the Database Architecture

The adopted database architecture provides several benefits:

- **Service Independence:** Each microservice owns and manages its own database table without depending on other services.
- **Loose Coupling:** Communication occurs through Kafka events rather than direct database access.
- **Scalability:** Individual services can be scaled independently based on transaction load.
- **Fault Isolation:** Failure of one service does not directly affect the databases of other services.
- **Maintainability:** Database schema changes remain localized to the owning service.
- **Data Integrity:** PostgreSQL transactions and constraints ensure reliable storage of payment information.
- **Traceability:** The audit database maintains a complete record of every transaction event for monitoring and compliance.
- **Extensibility:** New services and additional ledger tables can be introduced without impacting the existing database design.

2.6 Summary

The database architecture of the HPE APEX UPI Payment Switch is designed to support a scalable, event-driven microservices environment. PostgreSQL provides reliable transactional storage, while Kafka enables asynchronous communication between services. By following the database-per-service pattern, the architecture achieves high modularity, fault tolerance, and maintainability, making it suitable for processing high volumes of UPI transactions in a distributed payment system.

Chapter 3 – Database Design Principles

3.1 Overview

The database for the HPE APEX UPI Payment Switch is designed to provide reliable, secure, and efficient storage of transaction data. Since the application follows a microservice architecture, each service manages its own database table while communicating with other services through Apache Kafka events. This approach ensures data consistency, service independence, and easier maintenance.

3.2 Design Principles

The following principles are followed while designing the database:

1. Database per Service

Each microservice owns its respective database table. Services do not directly access tables owned by other services, reducing coupling and improving scalability.

2. Unique Transaction Identification

Every payment transaction is assigned a unique **Transaction ID (txn_id)**, which is used to track the transaction across all services and ledger tables.

3. Primary Keys

Each table contains an auto-generated **Primary Key (id)** to uniquely identify every record and maintain data integrity.

4. Data Consistency

Mandatory fields are defined using **NOT NULL** constraints to prevent incomplete transaction records. PostgreSQL constraints ensure valid and consistent data storage.

5. Event-Driven Updates

Database records are created and updated only after receiving Kafka events from the corresponding producer service. This keeps services loosely coupled and ensures reliable event processing.

6. Immutable Transaction Records

Once a transaction record is created, its core information such as transaction ID, payer, payee, and amount remains unchanged. Only fields such as **status** may be updated during the transaction lifecycle.

7. Audit and Traceability

All significant transaction events are recorded in the **audit_log** table, providing complete traceability for debugging, monitoring, reconciliation, and compliance purposes.

3.3 Benefits of the Design

The adopted database design provides the following advantages:

- Reduced dependency between microservices.
- Improved scalability and maintainability.
- Reliable transaction tracking using a unique transaction ID.
- Better data integrity through PostgreSQL constraints.
- Easier auditing and troubleshooting using centralized audit logs.
- Simplified future enhancements and schema modifications.

3.4 Summary

The database design principles ensure that the HPE APEX UPI Payment Switch remains scalable, reliable, and easy to maintain. By combining the database-per-service pattern with PostgreSQL's consistency features and Kafka's event-driven communication, the system supports efficient transaction processing while maintaining data integrity and traceability.

Chapter 4 – Database Schema Overview

4.1 Overview

The HPE APEX UPI Payment Switch uses a PostgreSQL database to store transaction and ledger information generated during the UPI payment lifecycle. Instead of maintaining a single shared table, the system separates data into multiple tables based on the responsibilities of each microservice. This modular schema improves maintainability, scalability, and fault isolation.

Each table is owned and managed by its corresponding service, while data synchronization between services is achieved through Apache Kafka events.

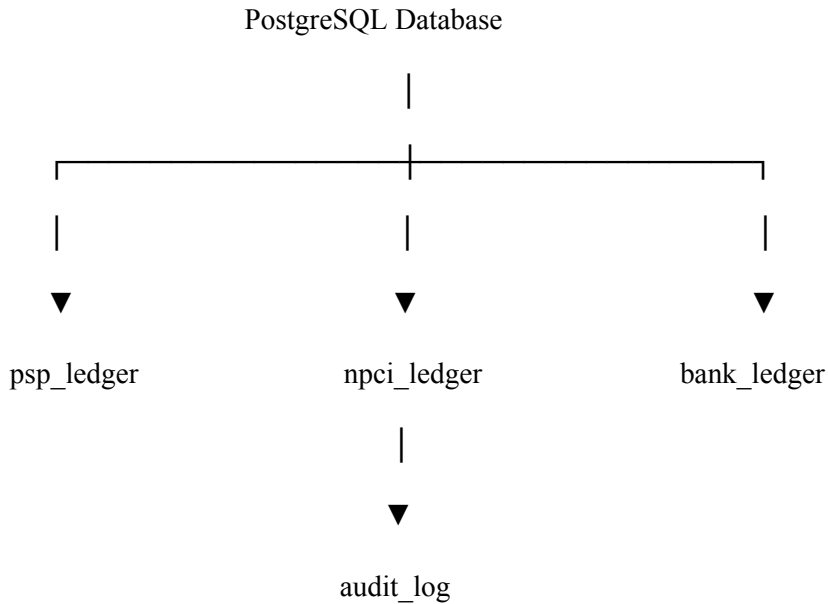
4.2 Database Schema

The PostgreSQL database consists of the following tables:

Table	Owner Service	Purpose
psp_ledger	PSP Ledger Service	Stores payment initiation details received from the PSP Switch.
npci_ledger	NPCI Ledger Service	Stores transaction routing, verification, and bank information.
bank_ledger	Bank Ledger Service	Stores debit and credit ledger entries after CBS processing.
audit_log	Audit Service	Stores the complete lifecycle and audit trail of every transaction.

4.3 Database Relationships

Although all tables belong to the same PostgreSQL database, they are logically independent. The **txn_id** acts as the common identifier that links records across all tables.



The Audit Service receives events from all microservices and stores them in the **audit_log** table, enabling complete transaction traceability.

4.4 Schema Characteristics

The database schema is designed with the following characteristics:

- Each table has an auto-generated primary key (**id**).
- **txn_id** is used as the common transaction identifier across services.
- Mandatory fields use **NOT NULL** constraints.
- Tables are independently managed by their respective microservices.
- Database communication between services occurs through Kafka events instead of direct table access.

4.5 Summary

The database schema is organized to support a distributed microservice architecture while maintaining reliable transaction storage. The separation of ledger tables improves modularity, and the common transaction identifier ensures that records belonging to the same payment can be traced across the entire UPI transaction lifecycle.

Chapter 5 – PSP Ledger Database Design

5.1 Overview

The **PSP Ledger Service** is responsible for storing the initial transaction details generated when a customer initiates a UPI payment. It acts as the first persistent storage layer in the payment processing pipeline and maintains the basic information required to track a transaction throughout its lifecycle.

The service consumes transaction initiation events published by the **PSP Switch** through Apache Kafka and stores them in the **psp_ledger** table.

5.2 Kafka Integration

Kafka Topic	Producer	Consumer
<code>upi.transactions.initiated</code>	PSP Switch	PSP Ledger Service

When a payment request is received, the PSP Switch validates the request and publishes an event to the Kafka topic. The PSP Ledger Service consumes this event and inserts the transaction details into the database.

5.3 Database Schema

```
CREATE TABLE psp_ledger (
  id SERIAL PRIMARY KEY,
  txn_id VARCHAR(100) NOT NULL,
  payer_vpa VARCHAR(100) NOT NULL,
  payee_vpa VARCHAR(100) NOT NULL,
  amount DOUBLE PRECISION NOT NULL,
  status VARCHAR(50) NOT NULL
);
```

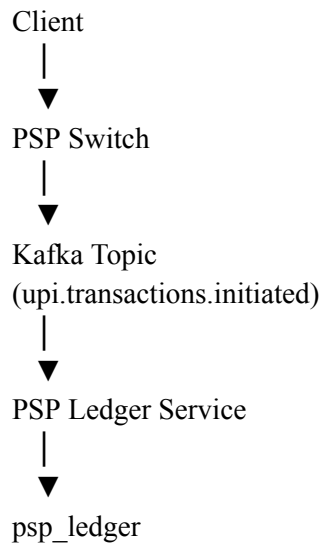
5.4 Column Description

Column	Description
id	Auto-generated primary key for each record.
txn_id	Unique transaction identifier used across all services.
payer_vpa	UPI ID of the customer initiating the payment.

payee_vpa	UPI ID of the beneficiary receiving the payment.
amount	Amount transferred during the transaction.
status	Current transaction status (e.g., INITIATED).

5.5 Data Flow

The PSP Ledger follows the workflow below:



5.6 Purpose and Benefits

The PSP Ledger provides the following benefits:

- Stores the initial payment request.
- Maintains a unique transaction record for tracking.
- Supports transaction monitoring and reconciliation.
- Enables downstream services to process validated transaction data.
- Provides reliable storage using PostgreSQL.

5.7 Summary

The **psp_ledger** table serves as the entry point for transaction persistence within the HPE APEX UPI Payment Switch. By storing payment initiation details and using Kafka-based event processing, the PSP Ledger Service ensures reliable and scalable handling of incoming UPI transactions before they proceed to verification and settlement stages.

Chapter 6 – NPCI Ledger Database Design

6.1 Overview

The **NPCI Ledger Service** stores transaction routing and verification details received from the NPCI Router. After the PSP Switch initiates a transaction, the NPCI Router validates the payment request, identifies the source and destination banks, and publishes the verification result through Apache Kafka.

The NPCI Ledger Service consumes these events and stores the routing information in the **npci_ledger** table. This data helps track the verification stage of every UPI transaction before it proceeds to bank processing.

6.2 Kafka Integration

Kafka Topic	Producer	Consumer
<code>upi.npci.verified</code>	NPCI Router	NPCI Ledger Service

After successful verification, the NPCI Router publishes an event to the Kafka topic, which is consumed by the NPCI Ledger Service for database storage.

6.3 Database Schema

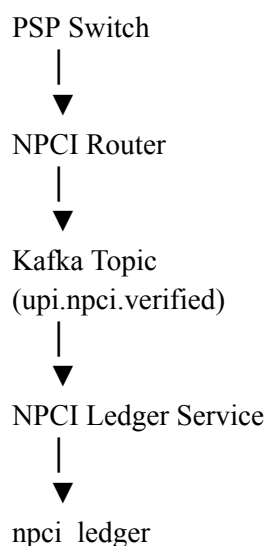
```
CREATE TABLE npci_ledger (
  id SERIAL PRIMARY KEY,
  txn_id VARCHAR(100) NOT NULL,
  source_bank VARCHAR(100) NOT NULL,
  destination_bank VARCHAR(100) NOT NULL,
  amount DOUBLE PRECISION NOT NULL,
  status VARCHAR(50) NOT NULL
);
```

6.4 Column Description

Column	Description
id	Auto-generated primary key for each record.
txn_id	Unique transaction identifier shared across all services.
source_bank	Bank from which the payment is initiated.

destination_bank	Beneficiary bank receiving the payment.
amount	Amount involved in the transaction.
status	Verification status of the transaction (e.g., VERIFIED, FAILED).

6.5 Data Flow



6.6 Purpose and Benefits

The NPCI Ledger provides the following benefits:

- Stores transaction verification results.
- Maintains source and destination bank details.
- Supports transaction routing and reconciliation.
- Enables downstream banking services to process verified transactions.
- Provides reliable storage for all NPCI-related transaction events.

6.7 Summary

The **npci_ledger** table acts as the routing and verification repository for the HPE APEX UPI Payment Switch. By storing verified transaction details received through Kafka, the NPCI Ledger Service ensures that only validated transactions proceed to the banking stage, improving the reliability and traceability of the overall payment process.

Chapter 7 – Bank Ledger Database Design

7.1 Overview

The **Bank Ledger Service** is responsible for storing the accounting entries generated after successful debit and credit operations performed by the Core Banking System (CBS). It acts as the financial ledger of the payment system by maintaining records of all completed banking transactions.

Once the CBS processes a transaction, it publishes debit and credit confirmation events through Apache Kafka. The Bank Ledger Service consumes these events and stores them in the **bank_ledger** table. For every successful UPI transaction, separate **Debit** and **Credit** entries are maintained to support reconciliation and financial reporting.

7.2 Kafka Integration

Kafka Topic	Producer	Consumer
<code>upi.cbs.debit.confirm</code>	CBS Service	Bank Ledger Service
<code>upi.cbs.credit.confirm</code>	CBS Service	Bank Ledger Service

The Bank Ledger Service listens to both Kafka topics and records the corresponding accounting entries in the database.

7.3 Database Schema

```
CREATE TABLE bank_ledger (
  id SERIAL PRIMARY KEY,
  txn_id VARCHAR(100) NOT NULL,
  account VARCHAR(100) NOT NULL,
  entry_type VARCHAR(20) NOT NULL,
  amount DOUBLE PRECISION NOT NULL,
```

status VARCHAR(50) NOT NULL

);

7.4 Column Description

Column	Description
id	Auto-generated primary key for each ledger record.
txn_id	Unique transaction identifier shared across all services.
account	Account number or UPI ID associated with the transaction.
entry_type	Indicates whether the entry is DEBIT or CREDIT .
amount	Amount processed during the transaction.
status	Current transaction status (e.g., SUCCESS, FAILED).

7.5 Data Flow

NPCI Router

|



CBS Service

|



Kafka Topics

(upi.cbs.debit.confirm)

(upi.cbs.credit.confirm)

|



Bank Ledger Service

|



bank_ledger

7.6 Purpose and Benefits

The Bank Ledger provides the following benefits:

- Stores debit and credit accounting entries.
- Maintains accurate financial transaction records.
- Supports reconciliation and settlement processes.
- Enables transaction tracking using a common **txn_id**.
- Ensures reliable and consistent storage using PostgreSQL.

7.7 Summary

The **bank_ledger** table serves as the financial record of all processed UPI transactions in the HPE APEX UPI Payment Switch. By consuming debit and credit confirmation events from the CBS Service, the Bank Ledger Service maintains accurate accounting information, ensuring reliable reconciliation, auditing, and settlement of payment transactions.

Chapter 8 – Audit Database Design

8.1 Overview

The **Audit Service** is responsible for maintaining a complete audit trail of every transaction processed by the HPE APEX UPI Payment Switch. Unlike the ledger services, which store data for specific stages of the payment process, the Audit Service records events generated throughout the entire transaction lifecycle.

It consumes events from multiple Kafka topics and stores them in the **audit_log** table. These records are used for transaction monitoring, troubleshooting, reconciliation, compliance, and performance analysis.

8.2 Kafka Integration

Kafka Topic	Producer	Consumer
<code>upi.transactions.initiated</code>	PSP Switch	Audit Service
<code>upi.npci.verified</code>	NPCI Router	Audit Service
<code>upi.cbs.debit.confirm</code>	CBS Service	Audit Service
<code>upi.cbs.credit.confirm</code>	CBS Service	Audit Service
<code>upi.cbs.reversal</code>	CBS Service	Audit Service

The Audit Service continuously listens to these Kafka topics and records every significant transaction event in the database.

8.3 Database Schema

```
CREATE TABLE audit_log (  
    id SERIAL PRIMARY KEY,  
    txn_id VARCHAR(100),  
    source VARCHAR(50),  
    status VARCHAR(50),  
    payer_vpa VARCHAR(100),  
    payee_vpa VARCHAR(100),  
    amount DOUBLE PRECISION,  
    stage VARCHAR(50),  
    remarks VARCHAR(255),  
    created_at TIMESTAMP  
);
```

8.4 Column Description

Column	Description
id	Auto-generated primary key for each audit record.
txn_id	Unique transaction identifier.
source	Service that generated the event (PSP, NPCI, CBS, etc.).
status	Current status of the transaction event.

payer_vpa UPI ID of the sender.

payee_vpa UPI ID of the beneficiary.

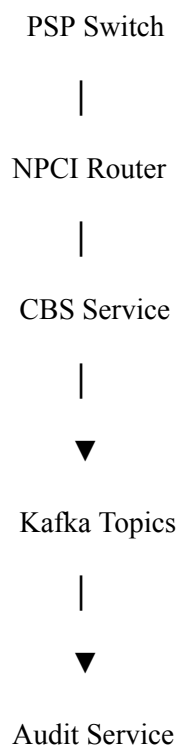
amount Transaction amount.

stage Processing stage of the transaction.

remarks Additional information or error details.

created_at Timestamp when the event was recorded.

8.5 Data Flow



|
▼
audit_log

8.6 Purpose and Benefits

The Audit Database provides the following benefits:

- Maintains a complete history of every transaction.
- Records events from all microservices in a centralized location.
- Supports debugging and failure analysis.
- Assists in transaction reconciliation and compliance.
- Enables monitoring of transaction status across different processing stages.
- Provides traceability for successful, failed, and reversed transactions.

8.7 Summary

The **audit_log** table acts as the centralized repository for all transaction events within the HPE APEX UPI Payment Switch. By collecting events from multiple Kafka topics, the Audit Service provides end-to-end visibility into the transaction lifecycle, enabling efficient monitoring, auditing, and troubleshooting while ensuring complete traceability of payment processing activities.

Chapter 9 – Data Flow

9.1 Overview

The HPE APEX UPI Payment Switch follows an **event-driven data flow** in which each microservice processes a transaction independently and stores its data in its own database table. Instead of directly accessing another service's database, services communicate through **Apache Kafka** by publishing and consuming events. This approach improves scalability, reliability, and fault isolation while ensuring that every stage of the transaction is recorded.

9.2 Transaction Data Flow

The following sequence illustrates how transaction data moves through the system:

Client

|



PSP Switch

|



Kafka Topic

(upi.transactions.initiated)

|



PSP Ledger Service

|



psp_ledger

|



NPCI Router

|



Kafka Topic

(upi.npci.verified)

|



NPCI Ledger Service

|



npci_ledger

|



CBS Service

|

└──────────────────▶ upi.cbs.debit.confirm

|

└──────────────────▶ upi.cbs.credit.confirm

|



Bank Ledger Service

|



bank_ledger

All Kafka Events

|



Audit Service

|



audit_log

9.3 Data Flow Description

The transaction begins when a customer initiates a payment through the **PSP Switch**. After validating the request, the PSP Switch publishes a transaction initiation event, which is stored by the **PSP Ledger Service**.

The transaction is then forwarded to the **NPCI Router** for routing and verification. Once verification is completed, the NPCI Router publishes a verification event, which is consumed by the **NPCI Ledger Service** and stored in the **npci_ledger** table.

Next, the verified transaction is processed by the **CBS Service**, which performs debit and credit operations. The CBS Service publishes confirmation events for both operations, and the **Bank Ledger Service** records them in the **bank_ledger** table.

Throughout the entire process, the **Audit Service** listens to all Kafka topics and stores every transaction event in the **audit_log** table, providing a complete history of the payment lifecycle.

9.4 Benefits of the Data Flow

The event-driven data flow offers several advantages:

- Eliminates direct database dependencies between services.
- Supports asynchronous communication through Kafka.
- Improves scalability by allowing services to process events independently.
- Enables fault isolation, ensuring one service failure does not affect others.
- Maintains complete transaction traceability using the **txn_id**.
- Provides reliable auditing through centralized event logging.

9.5 Summary

The data flow architecture ensures that every stage of a UPI transaction is processed independently and stored in the appropriate database table. Kafka enables efficient communication between microservices, while PostgreSQL provides reliable storage for ledger and audit data. This design supports high-throughput transaction processing, easy monitoring, and complete traceability across the payment lifecycle.

Chapter 10 – Constraints and Keys

10.1 Introduction

Database constraints and keys are essential for maintaining the integrity, consistency, and reliability of stored data. In the APEX UPI system, every table is designed with appropriate keys and constraints to ensure that transaction records remain valid throughout the payment lifecycle. These rules prevent duplicate entries, maintain accurate relationships between records, and ensure that mandatory transaction information is always available.

The use of constraints also improves database performance by enforcing standardized data storage and reducing the possibility of invalid or incomplete records.

10.2 Primary Keys

Each table contains a primary key that uniquely identifies every record stored in the database.

Table	Primary Key
psp_ledger	id
npci_ledger	id
bank_ledger	id
audit_log	id

The **id** column is defined as a **SERIAL** data type, allowing PostgreSQL to automatically generate unique values whenever a new record is inserted.

The primary key provides the following advantages:

- Ensures uniqueness of every record.
- Simplifies record retrieval.
- Improves indexing performance.

- Prevents duplicate primary key values.
- Supports efficient database operations.

10.3 Transaction Identifier

Although each table has its own primary key, the **Transaction ID (txn_id)** serves as the common identifier across all services.

A single transaction progresses through multiple services, and each service stores its own record using the same transaction ID.

Example:

Service	Transaction ID
PSP Ledger	TXN1001
NPCI Ledger	TXN1001
Bank Ledger	TXN1001
Audit Log	TXN1001

This approach allows developers to trace an individual transaction across different services without introducing direct database dependencies.

10.4 Database Constraints

Several constraints are applied to maintain data quality.

NOT NULL Constraint

Mandatory fields are configured using **NOT NULL** to ensure complete transaction records.

Examples include:

- Transaction ID
- Amount
- Status
- Account Information

Primary Key Constraint

Every table contains exactly one primary key.

This guarantees that every row is uniquely identifiable.

Data Type Constraints

Each column is assigned an appropriate data type.

Examples:

Data	Data Type
Transaction ID	VARCHAR
Amount	DOUBLE PRECISION
Status	VARCHAR
Timestamp	TIMESTAMP

Choosing suitable data types improves storage efficiency and query performance.

10.5 Data Integrity

The database design maintains data integrity by:

- Preventing duplicate records.
- Ensuring mandatory data is available.
- Maintaining transaction consistency.
- Preserving complete transaction history.

These mechanisms ensure that ledger and audit information remains reliable throughout the transaction lifecycle.

10.6 Summary

The database constraints and keys play a vital role in maintaining the consistency and reliability of transaction records. The combination of primary keys, transaction identifiers, mandatory fields, and appropriate data types provides a robust foundation for reliable financial data storage.

Chapter 11 – Backup and Recovery

11.1 Introduction

The PostgreSQL database stores important transaction, ledger, and audit information. Therefore, an effective backup and recovery strategy is essential to prevent data loss and ensure continuous availability of the payment system.

The backup strategy focuses on protecting transaction records, while the recovery process enables restoration of the database in case of hardware failures, software errors, or accidental data deletion.

11.2 Backup Strategy

The database should be backed up at regular intervals to preserve transaction history.

Recommended backup methods include:

- Daily full database backup.
- Weekly archive backup.
- Scheduled automated backups.
- Periodic backup verification.

Regular backups reduce the possibility of permanent data loss.

11.3 Backup Components

The backup process should include:

- Database schema.
- Table structures.
- Ledger records.
- Audit records.
- Transaction history.
- Configuration data.

Maintaining complete backups allows the system to be restored quickly whenever required.

11.4 Recovery Process

If the database becomes unavailable, the recovery process consists of the following steps:

1. Stop all dependent services.
2. Restore the latest database backup.
3. Verify table structures.
4. Validate transaction records.
5. Restart application services.
6. Resume transaction processing.

This approach minimizes downtime and ensures database consistency after restoration.

11.5 Backup Benefits

Implementing a proper backup strategy provides:

- Protection against data loss.
- Business continuity.
- Faster disaster recovery.
- Reliable transaction preservation.
- Improved system availability.

11.6 Summary

Backup and recovery procedures ensure that the PostgreSQL database remains available even during unexpected failures. Regular backups combined with systematic recovery procedures improve the reliability of the overall payment platform.

Chapter 12 – Future Enhancements

The current database design satisfies the functional requirements of the proposed APEX UPI system. However, several enhancements can be incorporated to improve scalability, security, and long-term maintainability.

Future improvements include:

Database Indexing

Additional indexes can be created on frequently searched columns such as **Transaction ID**, **Status**, and **Timestamp** to improve query performance.

Table Partitioning

As transaction volume increases, large tables can be partitioned based on transaction date or transaction status. This improves query execution and simplifies maintenance.

Database Replication

Database replication can be introduced to improve system availability and fault tolerance by maintaining synchronized copies of the PostgreSQL database.

Data Archiving

Older transaction records can be moved to archival storage while retaining recent transactions in the primary database. This improves overall database performance.

Data Encryption

Sensitive fields such as customer account information can be encrypted before storage to enhance data security.

Monitoring and Alerts

Database monitoring tools can be integrated to track performance metrics, storage utilization, and query execution time. Automated alerts can notify administrators about abnormal database behaviour.

Cloud Deployment

The PostgreSQL database can be migrated to managed cloud database services, providing automatic backup, scaling, and high availability.

Chapter 13 – Conclusion

The Database Design Document presented the logical and physical database design of the APEX UPI system. The database architecture was developed following the database-per-service principle, where each microservice independently manages its own data while communicating through Apache Kafka.

The PSP Ledger, NPCI Ledger, Bank Ledger, and Audit tables collectively provide complete transaction persistence and traceability. Each table is designed with appropriate keys, constraints, and data types to ensure reliable storage and efficient retrieval of transaction information.

The database design supports asynchronous event-driven processing by allowing each service to store transaction records independently without creating tight coupling between services. This improves maintainability, scalability, and fault isolation while simplifying future system enhancements.

The use of PostgreSQL provides reliable relational data storage with strong transactional consistency, making it suitable for financial applications. The combination of structured schema design, proper constraints, backup mechanisms, and scalable architecture ensures that the database can efficiently support high-volume UPI transaction processing.

Overall, the proposed database design establishes a robust foundation for maintaining ledger records, audit history, and transaction information throughout the payment lifecycle. It complements the microservice architecture of the APEX UPI platform and supports future expansion as additional services and transaction types are introduced.